Programmable Cost Functions for Robotic Trajectories

Repeated evaluation of explicit kinematics and trajectory constraints

Scientific working paper | Status: research proposal | 4 November 2026

Research opportunity

Robot planners must score many candidate configurations and trajectories against objectives and constraints. Some of the repeated computations are explicit trigonometric and polynomial expressions that could be evaluated in prepared batches. paceval UG proposes testing a mathematical execution layer for these selected functions, while leaving motion planning, rigid-body dynamics and geometric collision checking with specialized robotics software.



Figure 1. Proposed integration boundary for a reusable mathematical evaluation kernel.

1. Existing research tools and the exact gap to investigate

Pinocchio offers efficient rigid-body dynamics and analytical derivatives, while MoveIt 2 provides planning scenes and collision checking. Neither should be replaced by a generic formula engine. A meaningful paceval application would target narrow, explicit evaluation functions that appear many times within a planning loop, without silently changing geometry, collision rules or planner algorithms. [1,2]

2. Research contribution proposed by paceval

When many candidate joint states use a common mathematical structure, preconstructed execution graphs might help evaluate analytic costs and simple constraints efficiently. Such kernels could be useful for varying objectives between experiments and, over time, could be examined for batched FPGA processing.

paceval is a mathematical engine that transforms supported text expressions into reusable computation objects. Its published C/C++ API supports preparing calculations for repeated evaluation, including batched input sets. The initial study should use local library integration. Networked REST or MCP communication would distort latency measurements for the targeted arithmetic kernel. The product documentation is linked in the references.

3. Explicit mathematical test case

For a planar two-link arm with lengths l_1 and l_2, the end-effector position is x = l_1 cos(q_1) + l_2 cos(q_1+q_2) and y = l_1 sin(q_1) + l_2 sin(q_1+q_2). For a circular point-obstacle model with center (x_o,y_o) and radius r, g(q) = (x-x_o)^2 + (y-y_o)^2 - r^2 >= 0 ensures only that the end-effector point lies outside the circle. It does not prove full-arm collision freedom.

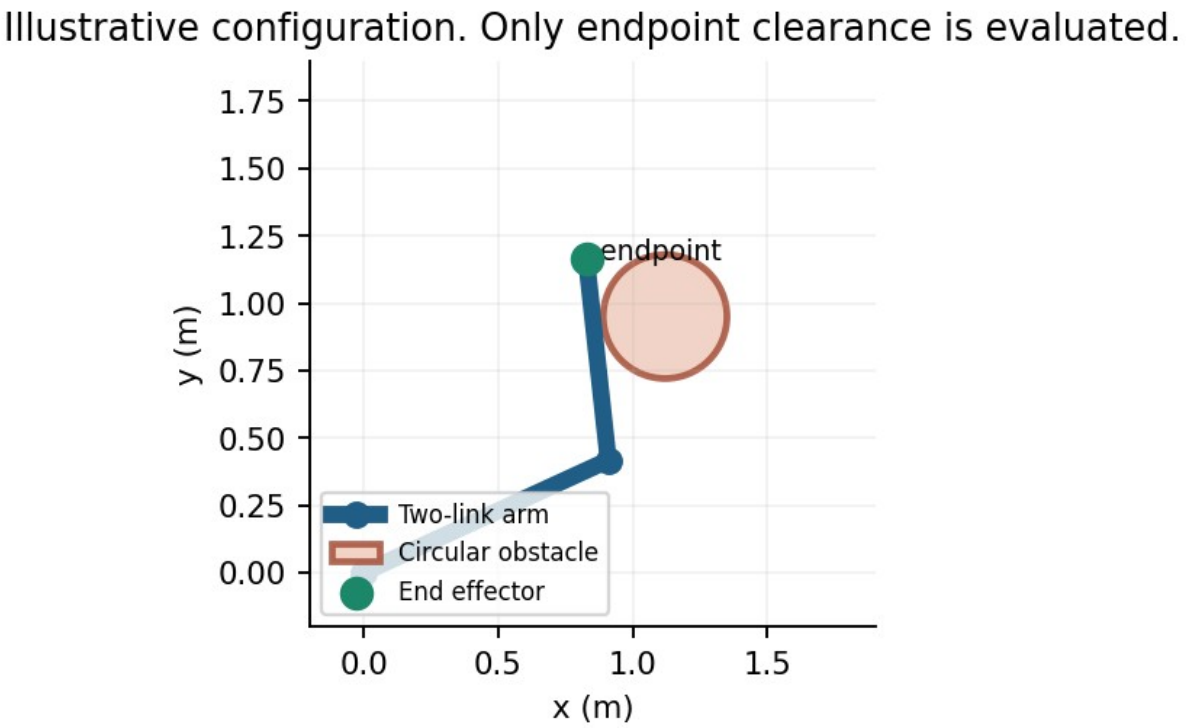


Figure 2. Two-link planar robot and endpoint clearance example. Synthetic or analytic example, not a paceval performance measurement.

The figure shows a synthetic two-link geometry and the restricted endpoint clearance test. It intentionally distinguishes the endpoint from the complete robot links. Full robot collision checking must remain with a geometric planner such as MoveIt 2.

4. Reproducible pilot experiment

Evaluate a planar reference arm and then a documented multi-joint configuration. Compute identical candidate costs in optimized C++ and paceval for matching joint-angle batches. Compare with independent forward-kinematics results, explore singular and near-boundary configurations and then measure any difference in total planning time. Keep the planner search and full geometric collision checking unchanged.

Evaluation dimension	Study design
Mathematical correctness	Identical formulas and input domain. Independent numerical references, including boundary cases.
Kernel efficiency	Candidate evaluations per second; per-cost accuracy; complete planning time; geometric constraint feasibility; worst observed latency; memory traffic and FPGA resource estimates.
Reproducibility	Thirty or more repetitions after warm-up, published inputs and software versions. Report median, dispersion and uncertainty.
Full-system validity	Include graph creation, adapters, data movement and the effect on total execution time.

5. Integration with established scientific software

Add an optional mathematical callback for selected scalar objectives and analytically defined bounds. Keep robot state representation, planning graph, collision geometry and dynamics in Pinocchio and MoveIt 2. If derivatives are needed, retain independently validated analytic or automatic-differentiation implementations.

6. Programmable FPGA and ASIC perspective

A future FPGA pipeline might evaluate many fixed-form candidate costs in parallel. To establish utility, benchmark scheduling overhead, candidate generation, collision geometry, data transfer and overall planning quality, not only arithmetic throughput.

7. Expected scientific contribution

The result would be a reusable reference suite for analytically defined planning costs and safety proxies, with a clear boundary between mathematical scoring and full robot-geometry validation.

8. Open invitation to collaborate

paceval UG invites robotics researchers to jointly benchmark programmable mathematical scoring for trajectories, including reproducible datasets and optional future work on larger combined execution graphs.

paceval UG is prepared to make the existing mathematical technology available for jointly defined research and to discuss targeted API extensions, open benchmark artifacts and integration formats with the scientific partners. Publication rights, licensing and intellectual property would be agreed before a joint project begins.

9. Practical work packages

First, validate planar kinematics and endpoint constraints against independent analytic references. Next, integrate an optional batched score callback into a robot planner while keeping full collision checking unchanged. Finally, repeat on a documented multi-joint test and report whole-planner effects.

Scientific and technical sources

[1] Pinocchio rigid-body algorithms and derivatives. Open source 1

[2] MoveIt 2 planning scene and collision checking. Open source 2

[3] MoveIt 2 collision checking tutorial. Open source 3

[4] paceval C/C++ API. Open source 4

Technical information about paceval: paceval.com

Research contact: info@paceval.com